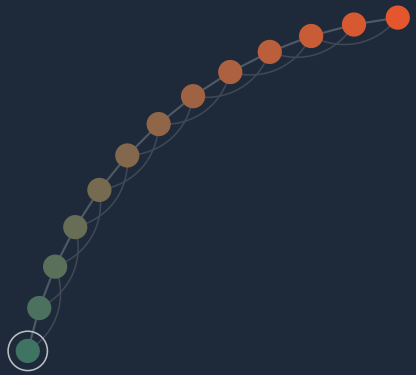
E1 277 · COURSE PROJECT · PAPER #22

# Least-Squares Temporal Difference Learning

Reimplementing Boyan (ICML 1999)
and a ridge-regularised extension

**Member A** · **Member B** · **Member C** · **Member D**

E1 277 Reinforcement Learning · November 2026



# What we did

**One-line summary.** TD($\lambda$) solves a linear system by slow, noisy gradient steps. LSTD($\lambda$) estimates that system from data and **solves it directly**: same answer, less data, no step size.

**Talk outline**


1 Motivation & setup — Member A
2 TD($\lambda$) and its weaknesses — Member B
3 The LSTD($\lambda$) algorithm — Member C
4 Experiments & novelty — Member D


| Course task | Our work |
|---|---|
| Understand algorithm | Derivation + paper's equivalences proved as unit tests |
| Run authors' code | None public (1999) $\Rightarrow$ full reimplementation, matches paper Figs. 5–6 |
| Change settings | $\lambda$, 12 TD step sizes, feature dim. $d$ |
| New environment | 19-state random walk, weak features |
| Improvement | Ridge LSTD for very few trajectories |

# 01

# Motivation & setup

Member A

Member A

# Policy evaluation

- Fixed policy $\pi$ in an MDP $\Rightarrow$ a Markov chain with rewards.
- **Value function**: expected total future reward from each state

$$V^{\pi}(s) = \mathbb{E}\Big[ \textstyle\sum_{t \geq 0} \gamma^t r_t \;\Big|\; s_0 = s \Big]$$

- Inner loop of policy iteration, actor–critic, STAGE, …
- If the transition matrix $P$ is known, it is just a linear system (Bellman equation):

$$V = r + \gamma P V$$

- **In practice:** (1) $P$ is unknown — we only see sampled trajectories; (2) too many states to store a table.

Member A

# Linear function approximation

- Describe each state by $K$ features $\boldsymbol{\phi}(s) \in \mathbb{R}^K$ and learn only $K$ weights:

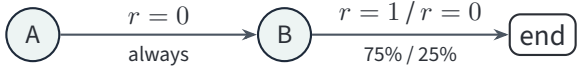
$$\hat{V}(s) = \boldsymbol{\phi}(s)^\top \boldsymbol{\theta}$$

- Covers lookup tables, state aggregation, tile coding / CMACs, RBFs, polynomials.
- **Data:** trajectories of transitions $(s_t,\ r_t,\ s_{t+1})$ until termination.
- **Goal:** find $\boldsymbol{\theta}$ such that $\hat{V} \approx V^\pi$, using as few trajectories as possible.
- **Metric (paper):** RMS error over all states

$$\mathrm{RMSE} = \sqrt{\tfrac{1}{|S|} \textstyle\sum_s \left(\boldsymbol{\phi}(s)^\top \boldsymbol{\theta} - V^\pi(s)\right)^2}$$

Member A

# Monte Carlo vs. TD: the intuition




**Data:** 8 episodes. One starts at A and is unlucky (A→B→0). Seven start at B: six pay 1, one pays 0.

**MC**: target = actual return.
"From A I once got 0" $\Rightarrow \hat{V}(A) = 0$.

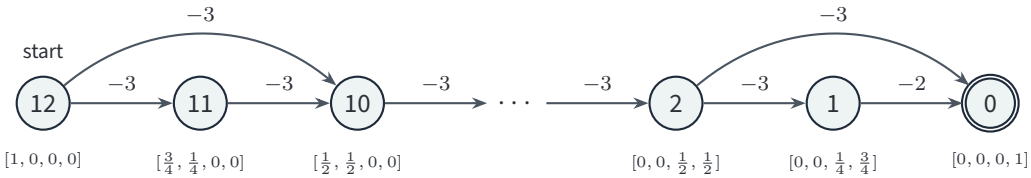
**TD**: target = $r + \hat{V}(\text{next})$.
"A gives 0 then B, and B averages 0.75"
$\Rightarrow \hat{V}(A) = 0.75$.

| | True | MC | TD |
|---|---|---|---|
| $V(B)$ | 0.75 | 0.75 | 0.75 |
| $V(A)$ | 0.75 | **0** | **0.75** |

MC: unbiased, *high variance*.
TD: *bootstraps* from its own estimate; low variance, biased while estimates are wrong.

Member A

# Test domain: the Boyan chain (paper Fig. 4)




- 13 states, start at 12, state 0 terminal, $\gamma = 1$. From $s \geq 2$: go to $s-1$ or $s-2$ (prob. $\frac{1}{2}$ each), reward $-3$.
- $K = 4$ features: anchors at 12, 8, 4, 0; linear interpolation in between.
- True values $V^\pi(s) = -2s$ are **exactly representable**: $\boldsymbol{\theta}^* = (-24, -16, -8, 0)$.
- Every constant above checked against the paper and covered by unit tests.

# 02

# TD($\lambda$)

Member B

Member B

# From TD(0) to TD($\lambda$)

**TD(0)**: nudge $\boldsymbol{\theta}$ toward a one-step target

$$\delta_t = r_t + \gamma\, \boldsymbol{\phi}(s_{t+1})^\top \boldsymbol{\theta} - \boldsymbol{\phi}(s_t)^\top \boldsymbol{\theta}, \qquad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha\, \delta_t\, \boldsymbol{\phi}(s_t)$$

**TD($\lambda$)**: credit each TD error to *all recent* states via an eligibility trace

$$\mathbf{z}_t = \gamma\lambda\, \mathbf{z}_{t-1} + \boldsymbol{\phi}(s_t), \qquad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha\, \delta_t\, \mathbf{z}_t$$

- $\lambda = 0$: one-step TD (low variance, biased)
- $\lambda = 1$: Monte Carlo (unbiased, high variance)
- In between: trades bias for variance

- $\mathbf{z}_t$ propagates reward backwards *without storing the trajectory*
- Cost: $O(K)$ per step

Member B

# TD(λ) in the paper (Fig. 1) and step sizes

**TD(λ), offline (paper Fig. 1)**

for trajectory $n = 1, 2, \ldots$
  $\Delta \leftarrow 0$, $\mathbf{z} \leftarrow \boldsymbol{\phi}(s_0)$
  for each step $t$:
    $\Delta \leftarrow \Delta + \mathbf{z}\left(r_t + (\boldsymbol{\phi}_{t+1} - \boldsymbol{\phi}_t)^\top \boldsymbol{\theta}\right)$
    $\mathbf{z} \leftarrow \lambda \mathbf{z} + \boldsymbol{\phi}_{t+1}$
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha_n \Delta$

$\boldsymbol{\phi}(\text{end}) = 0$; $\boldsymbol{\theta}$ is fixed within a trajectory.

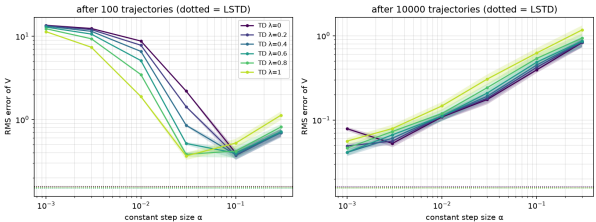
**Step-size schedule (paper Eq. 8)**

$$\alpha_n = a_0 \frac{n_0 + 1}{n_0 + n}$$

- $a_0 \in \{0.1,\ 0.01\}$
- $n_0 \in \{10^2,\ 10^3,\ 10^6\}$
- $\Rightarrow$ 6 schedules (paper)
- **We add** 6 constant $\alpha \in \{0.001, \ldots, 0.3\}$
- $\Rightarrow$ 12 TD configurations per $\lambda$

Member B

# Weakness of TD: the step size




**No single $\alpha$ is good both early and late** ($\lambda = 0.4$, RMSE):

| | $n = 100$ | $n = 10^4$ |
|---|---|---|
| TD $\alpha = 0.3$ | 0.70 | 0.85 |
| TD $\alpha = 0.001$ | 13.2 | 0.041 |
| TD, best sched. | 0.31 | 0.036 |
| **LSTD** | **0.15** | **0.016** |

Other weaknesses (paper): sensitive to initial $\boldsymbol{\theta}$ and to feature scaling; each sample used once, then forgotten.

# 03

# LSTD($\lambda$)

Member C

Member C

# Where does TD($\lambda$) converge?

- Averaged over trajectories, one TD($\lambda$) pass changes $\boldsymbol{\theta}$ by

$$\mathbb{E}[\Delta\boldsymbol{\theta}] = \alpha\,(\mathbf{d} + C\boldsymbol{\theta}), \quad \mathbf{d} = \mathbb{E}\Big[\textstyle\sum_t \mathbf{z}_t r_t\Big], \quad C = \mathbb{E}\Big[\textstyle\sum_t \mathbf{z}_t\big(\gamma\boldsymbol{\phi}_{t+1} - \boldsymbol{\phi}_t\big)^\top\Big]$$

- TD stops moving when

$$\boxed{\mathbf{d} + C\boldsymbol{\theta} = 0}$$

- $C$ is negative definite $\Rightarrow$ unique fixed point (Tsitsiklis & Van Roy).
- **TD is stochastic gradient descent on a $K \times K$ linear system** — it never writes the system down.

> **Idea:** estimate $\mathbf{d}$ and $C$ from data and solve the system directly.

Member C

# The LSTD($\lambda$) algorithm (paper Fig. 2)

**LSTD($\lambda$)**

$A \leftarrow 0, \mathbf{b} \leftarrow 0$
for each trajectory:
  $\mathbf{z} \leftarrow 0$
  for each step $t$:
    $\mathbf{z} \leftarrow \gamma\lambda\mathbf{z} + \boldsymbol{\phi}_t$
    $A \leftarrow A + \mathbf{z}\,(\boldsymbol{\phi}_t - \gamma\boldsymbol{\phi}_{t+1})^\top$
    $\mathbf{b} \leftarrow \mathbf{b} + \mathbf{z}\,r_t$
whenever needed: $\boldsymbol{\theta} = A^{-1}\mathbf{b}$

- After $n$ trajectories: $\mathbf{b} \approx n\mathbf{d}$, $A \approx -nC$
- $\Rightarrow A^{-1}\mathbf{b}$ estimates TD's fixed point
- **No step size, no initial $\theta$**
- Invariant to feature scaling
- Keeps *all* data in $A$, $\mathbf{b}$
- Singular $A$ (too little data): paper uses SVD; we use `solve` with an SVD fallback

Member C

# Three views of LSTD($\lambda$) (paper Secs. 2–3)

1. $\lambda = 0$: exactly Bradtke & Barto's LSTD (1996), with a much simpler derivation.
2. $\lambda = 1$: least-squares regression of features onto Monte-Carlo returns, built *incrementally* (telescoping: $A = \sum_t \boldsymbol{\phi}_t \boldsymbol{\phi}_t^\top$, $\mathbf{b} = \sum_t \boldsymbol{\phi}_t G_t$).
3. **Model-based view:** with one feature per state, LSTD(0) accumulates

$$A = N - C_{\text{count}}, \quad \mathbf{b} = \mathbf{s} \;\Rightarrow\; \boldsymbol{\theta} = (N - C_{\text{count}})^{-1}\mathbf{s},$$

which is exactly **"count transitions, build the empirical Markov chain, solve its Bellman equations"**.

**Corridor example, one pass over the 8 episodes**

LSTD(0): $\hat{V}(A) = 0.75$ (= TD's limit). LSTD(1): $\hat{V}(A) = 0$ (= MC). TD(0) needs $\sim$2000 replays with $\alpha = 0.01$ to reach 0.75.

All three equivalences are checked by unit tests in our repo.

Member C

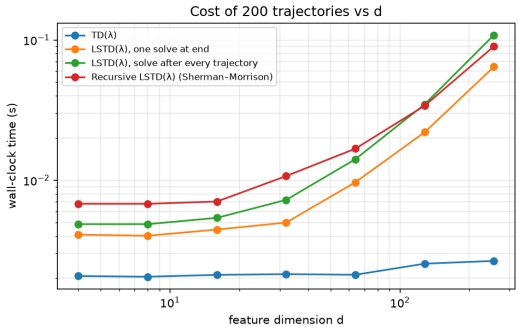
# Computational cost and recursive LSTD

| | per step | per solve |
|---|---|---|
| TD($\lambda$) | $O(K)$ | — |
| LSTD($\lambda$) | $O(K^2)$ | $O(K^3)$ |
| Recursive LSTD | $O(K^2)$ | — |

**Recursive LSTD** keeps $B = A^{-1}$ up to date (Sherman–Morrison), $\mathbf{u} = \boldsymbol{\phi}_t - \gamma\boldsymbol{\phi}_{t+1}$:

$$B \leftarrow B - \frac{B\mathbf{z}\,\mathbf{u}^\top B}{1 + \mathbf{u}^\top B\mathbf{z}}$$

Start $B_0 = \delta I \Leftrightarrow$ ridge with $\epsilon = 1/\delta$.




At $d = 256$: TD 2.7 ms vs. LSTD 64 ms (200 trajectories).

Member C

# Experimental setup

**Reproduction (paper Sec. 4)**

- Boyan chain, $\lambda \in \{0, 0.2, \ldots, 1\}$
- Up to $10^4$ trajectories
- LSTD vs. 12 TD configurations
- RMSE over all 13 states
- **20 seeds** (paper: 10), mean $\pm$ 1 SE
- Same episodes for every algorithm within a seed (common random numbers)

**Implementation**

- Python + NumPy, CPU only
- One module per member, frozen interfaces
- 47 unit tests, incl. paper's equivalences
- Config-driven runner, full sweep $\approx$ 30 s on a laptop

**Fair comparison:** TD is shown with its *best* step size — even an oracle that picks the best of 12 configurations separately at every $n$.

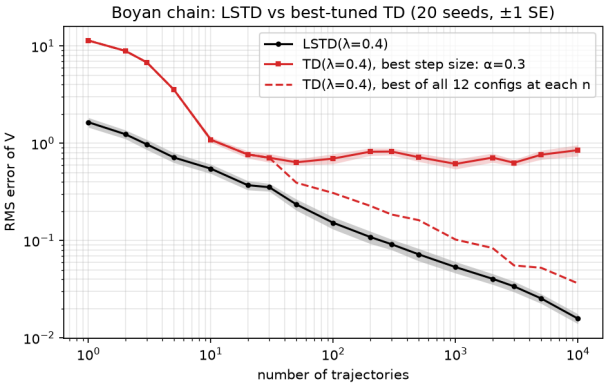
# 04

# Experiments & novelty

Member D

Member D

# Headline result: LSTD is more data-efficient




**$1.64 \rightarrow 0.016$**

LSTD RMSE from 1 to $10^4$ trajectories

**$\approx 0.7$**

where the best single TD $\alpha$ (0.3) plateaus

**$\approx 2\times$**

lower error than TD with its *best* $\alpha$ *picked at each* $n$ (dashed)

**3–4×**

fewer trajectories for the same error

Member D

# Decaying step sizes and every $\lambda$ (paper Figs. 5–6)

**Schedule** = how TD's step size $\alpha$ changes over time; here, 6 *decaying* ones. **Black = LSTD.**
$a_0$ = starting step size, $n_0$ = how long before $\alpha$ starts decaying ($\alpha$ halves after $\approx n_0$ traj.).

**Fig. 5** learning curves at $\lambda = 0.4$

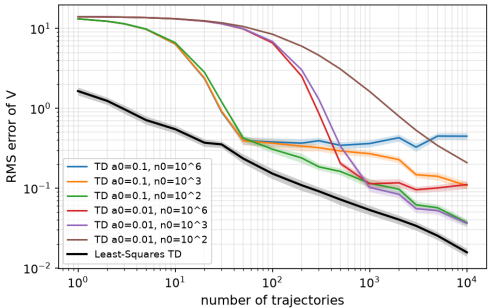
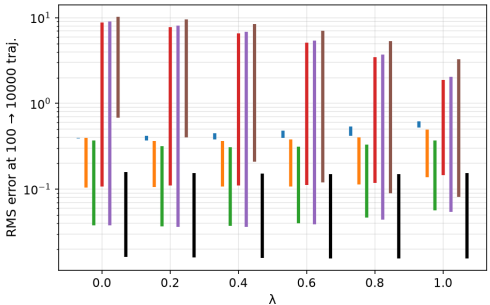


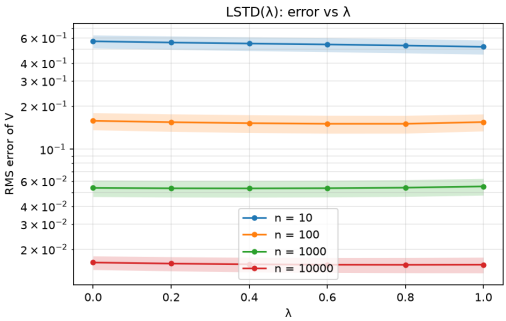

**Fig. 6** every $\lambda$: bar = 100 $\rightarrow 10^4$ traj.




**Same conclusion as the paper:** LSTD is lowest for every schedule and every $\lambda$.

Member D

# Effect of $\lambda$




| | $n = 100$ | | $n = 10^4$ | |
|---|---|---|---|---|
| $\lambda$ | LSTD | TD$^\star$ | LSTD | TD$^\star$ |
| 0.0 | **0.158** | 0.368 | **0.016** | 0.038 |
| 0.2 | **0.155** | 0.318 | **0.016** | 0.036 |
| 0.4 | **0.152** | 0.308 | **0.016** | 0.036 |
| 0.6 | **0.151** | 0.312 | **0.016** | 0.039 |
| 0.8 | **0.151** | 0.329 | **0.016** | 0.044 |
| 1.0 | **0.155** | 0.366 | **0.016** | 0.054 |

TD$^\star$ = best of 12 TD configs for that $n$ and $\lambda$.

- $\lambda$ barely matters for LSTD: features are *exact*, so every $\lambda$ converges to the same $\boldsymbol{\theta}^* = (-24, -16, -8, 0)$
- LSTD beats TD$^\star$ by $2$–$3.5\times$ everywhere

Member D

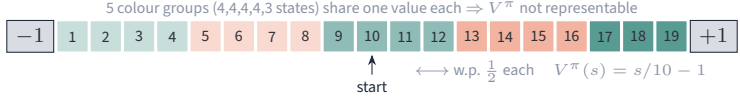
# Novelty: ridge LSTD for very few trajectories

**Problem.** With 1–10 trajectories, $A$ can be singular or ill-conditioned $\Rightarrow \boldsymbol{\theta} = A^{-1}\mathbf{b}$ is unstable.
**Method.** Ridge LSTD: solve $(A + \epsilon I)\,\boldsymbol{\theta} = \mathbf{b}$ (= recursive LSTD with $B_0 = I/\epsilon$).
**Questions.**

**Q1** How much $\epsilon$ should we add?

**Q2** With 1–5 trajectories, can it match the best-tuned TD?

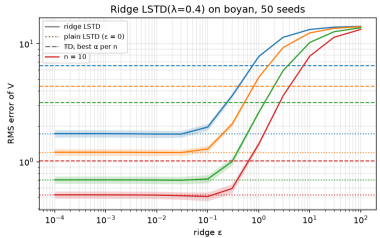
**Q3** Does it help on a harder problem with **weak features**: the **19-state random walk** (S&B Ex. 7.1)?



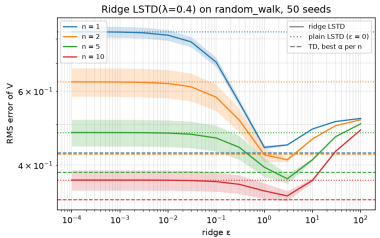

**Setup.** Both environments, 50 seeds, $\lambda = 0.4$, $n \in \{1, 2, 5, 10\}$, 13 values of $\epsilon$ from $10^{-4}$ to $10^{2}$; same episodes for all methods. TD reference = best of 5 constant $\alpha$ (0.01–1) at each $n$.

Member D

# Novelty: results




Boyan chain (exact features)




19-state random walk (weak features)

**Q1** **A sweet spot:** $\epsilon \approx 0.03$–$0.1$ (Boyan), $\epsilon \approx 1$–3 (random walk); larger $\epsilon$ shrinks $\boldsymbol{\theta} \to 0$ and hurts.

**Q2** **Nearly:** random walk, $n = 1$: 0.44 vs. best TD 0.43; better than TD at $n = 2, 5$. (Boyan: plain LSTD already wins.)

**Q3** **Helps most here:** error **halved** at $n = 1$ (0.83 $\to$ 0.44). Boyan: at most 3%.

Member D

# Conclusions and limitations

## Conclusions

- **Reproduced the paper:** LSTD beats TD at every $n$, $\lambda$ and step size ($2$–$3.5\times$ below TD with the best $\alpha$ at each $n$).
- TD's best $\alpha$ depends on the data budget; a bad $\alpha$ costs $10$–$100\times$.
- LSTD: no step size, insensitive to $\lambda$ here.
- Price: $O(K^2)$ per step vs. $O(K)$.
- Ridge helps when data are scarce *and* features weak.

## Limitations

- Toy chains; Boyan features are exact (best case for LSTD).
- TD tuned over a 12-point grid only.
- Fixed $\epsilon$ weakens as $n$ grows ($A$ is a sum).
- Ridge shrinks toward $\boldsymbol{\theta} = 0$, not a prior.

# Thank you

Questions?

src/ tests/ results/figures/

# Backup: likely questions

**Why more data-efficient?** It keeps every transition in $A, \mathbf{b}$ and solves exactly. TD uses each sample for one small step and then forgets it. LSTD is effectively model-based.

**What if $A$ is singular?** Too little data or redundant features. Paper: SVD pseudo-inverse. Us: `solve` with SVD fallback, or ridge $(A + \epsilon I)$.

**Cost in $K$?** $O(K^2)$ per step, $O(K^3)$ per solve (or $O(K^2)$ per step with recursive LSTD), vs. $O(K)$ for TD. Many features + cheap data $\Rightarrow$ TD can win on wall-clock time.

**LSTD with $\lambda = 1$?** Linear regression of features onto Monte-Carlo returns.

**Control?** Yes: LSTD-Q inside policy iteration = LSPI (Lagoudakis & Parr, 2003).

**Is the comparison fair?** Same episodes per seed; TD gets its best of 12 configs at every $n$.

**Why does ridge not help on Boyan?** $A$ is well conditioned, and shrinkage biases $\boldsymbol{\theta}$ toward 0 while $\boldsymbol{\theta}^* = (-24, -16, -8, 0)$.

# Backup: why LSTD(1) = Monte-Carlo regression

With $\lambda = \gamma = 1$, $\mathbf{z}_t = \sum_{i \le t} \boldsymbol{\phi}_i$ and $\boldsymbol{\phi}_T = 0$ at termination:

$$A = \sum_t \mathbf{z}_t (\boldsymbol{\phi}_t - \boldsymbol{\phi}_{t+1})^\top = \sum_i \boldsymbol{\phi}_i \sum_{t \ge i} (\boldsymbol{\phi}_t - \boldsymbol{\phi}_{t+1})^\top = \sum_i \boldsymbol{\phi}_i \boldsymbol{\phi}_i^\top$$

$$\mathbf{b} = \sum_t \mathbf{z}_t r_t = \sum_i \boldsymbol{\phi}_i \sum_{t \ge i} r_t = \sum_i \boldsymbol{\phi}_i G_i$$

So $\boldsymbol{\theta} = A^{-1}\mathbf{b}$ is ordinary least squares of $G_i$ on $\boldsymbol{\phi}_i$ — built incrementally without storing trajectories.

# Backup: code $\leftrightarrow$ paper

| Paper | Code | Owner |
|---|---|---|
| Fig. 4 chain and features | `env.BoyanChain`, `features.boyan_features` | A |
| Exact $V^\pi$ | `env.true_values` (Bellman solve) | A |
| Fig. 1 TD($\lambda$), Eq. 8 | `td_lambda.td_lambda_curve`, `step_size` | B |
| Fig. 2 LSTD($\lambda$) | `lstd.lstd_lambda_curve`, `solve_theta` | C |
| Recursive LSTD (Sec. 2) | `lstd.recursive_lstd` | C |
| Sec. 3 model-based view | `tests/test_lstd.py` | C |
| Figs. 5, 6 and our figures | `run_experiment.py`, `plot_results.py` | D |

```
python -m src.run_experiment && python -m src.plot_results
```